·Article·

Interactive hepatic parenchymal transection simulation with haptic feedback

Hongyu WU[1,2*], Haonan YU[1], Fan YE[1], Jian SUN[3], Yuan GAO[3], Ke TAN[3], Aimin HAO[1]

1. *State Key Laboratory of Virtual Reality Technology and Systems, School of Computer Science and Engineering, Beihang University, Beijing* 100191*, China*

2. *Beijing Advanced Innovation Center for Biomedical Engineering, Beijing* 100191*, China*

3. *Chinese PLA General Hospital, Beijing* 100039*, China*

*** Corresponding author,** whyvrlab@buaa.edu.cn

Received: 11 March 2021 **Revised:** 26 August 2021 **Accepted:** 6 September 2021

Supported by NSFC (61902014); Research Unit of Virtual Human and Virtual Surgery, Chinese Academy of Medical Sciences (2019RU004); Medical Innovation Research Project of PLA General Hospital (CX19032).

Citation: Hongyu WU, Haonan YU, Fan YE, Jian SUN, Yuan GAO, Ke TAN, Aimin HAO. Interactive hepatic parenchymal transection simulation with haptic feedback. Virtual Reality & Intelligent Hardware, 2021, 3(5): 383—396
DOI: 10.1016/j.vrih.2021.09.003

Abstract **Background** Liver resection involves surgical removal of a portion of the liver. It is used to treat liver tumors and liver injuries. The complexity and high-risk nature of this surgery prevents novice doctors from practicing it on real patients. Virtual surgery simulation was developed to simulate surgical procedures to enable medical professionals to be trained without requiring a patient, a cadaver, or an animal. Therefore, there is a strong need for the development of a liver resection surgery simulation system. We propose a real-time simulation system that provides realistic visual and tactile feedback for hepatic parenchymal transection. **Methods** The tetrahedron structure and cluster-based shape matching are used for physical model construction, topology update of a three-dimensional liver model soft deformation simulation, and haptic rendering acceleration. During the liver parenchyma separation simulation, a tetrahedral mesh is used for surface triangle subdivision and surface generation of the surgical wound. The shape-matching cluster is separated via component detection on an undirected graph constructed using the tetrahedral mesh. **Results** In our system, cluster-based shape matching is implemented on a GPU, whereas haptic rendering and topology updates are implemented on a CPU. Experimental results show that haptic rendering can be performed at a high frequency (>900Hz), whereas mesh skinning and graphics rendering can be performed at 45fps. The topology update can be executed at an interactive rate (>10Hz) on a single CPU thread. **Conclusions** We propose an interactive hepatic parenchymal transection simulation method based on a tetrahedral structure. The tetrahedral mesh simultaneously supports physical model construction, topology update, and haptic rendering acceleration.

Keywords Virtual surgery; Hepatic parenchymal transection; Position-based dynamics

1 Introduction

Liver resection, also known as hepatectomy, involves the surgical removal of all or a portion of the liver. It

is used to treat liver tumors and liver injuries[1]. Hepatic parenchymal transection is necessary during hepatectomy. In this procedure, the liver parenchyma is isolated via blunt separation or electrocoagulation, and the hepatic duct system, including the bile duct, hepatic artery, hepatic vein, and portal vein, can remain uninjured from surgical instruments. This typically necessitates skillful surgical performance because inappropriate treatment can result in potentially life-threatening complications. The complexity and high risk of hepatic parenchymal transection prevents novice doctors from performing the procedure on real patients. Virtual surgery, based on virtual reality technology, is performed to train surgeons without using animals or cadavers to gain experience prior to performing surgery on live patients[2]. This approach has the potential to improve the skills of surgeons without jeopardizing the life of patients.

This paper presents a system for hepatic parenchymal transection simulation with plausible visual and tactile feedback. Novice doctors can experience the operation procedure through our system. For this system, we used a GPU accelerated position-based dynamics framework to simulate liver deformation, and the shape matching[3,4] constraint was applied for shape preservation and haptic collision detection. A constraint-based method was used to calculate the position of the graphical tool in the haptic rendering cycle. The tetrahedron structure generated from the isotropic meshes was used for the physical model construction and topology update of the liver. Our system can realize interactive and arbitrary separation simulations. The fragments generated during separation were effectively detected and eliminated. The contributions of this study are as follows:

- A framework for hepatic parenchymal transection simulation based on tetrahedron is developed.
- Haptic rendering of a rigid tool and soft body interactions based on shape matching is performed.

The remainder of this paper is organized as follows. In Section 2, we briefly review the related studies. Section 3 presents our method, including the framework of our system, modeling of the liver system, topology updating for the liver model, and haptic rendering. Section 4 presents the experiments conducted in this study. Section 5 presents a discussion of the results.

2 Related work

Soft tissue deformation, electrocautery, and haptic rendering are the three essential components of surgery simulation. Next, we briefly review the studies associated with these components.

2.1 Soft tissue deformation simulation

The simulation of volumetric soft bodies is an important research topic in computer graphics. The mass spring system[5] is widely used for cloth simulations owing to its simplicity. In this approach, the soft body is modeled as a set of point masses connected by ideal weightless springs. However, it does not preserve the volume of the soft body well when deformation occurs, and the stiffness parameters of the springs cannot be tuned easily. The finite element method[6], a more physically accurate approach, is used to solve partial differential equations that govern the dynamics of an elastic material. The soft body is modeled as an elastic continuum by partitioning it into a number of solid elements and solving for the stresses and strains in each element. However, it cannot achieve real-time performance owing to its high computational complexity. Position-based dynamics[7] (PBDs) have become popular in the graphics community. Unlike impulse/velocity-based approaches, PBDs directly manipulate positions. This presents many advantages, such as the avoidance of overshooting problems in explicit integration schemes and easier management of collision constraints.

2.2 Electrocautery simulation

Electrocauterization is the process of destroying tissues (or cutting through soft tissues) using heat conduction from a metal probe heated by an electric current. Volumetric data are used for modeling soft tissues, and the removal of tissues can be simulated using marching cube-based algorithms[8]. Pan et al. used a heat transfer function to simulate the thermal transmission[9]. They modeled the physical characteristics of soft tissues using two types of particles, i.e., physical particles with lower resolutions for meshless deformations, and graphical particles with higher resolutions for thermal transmission and mesh reconstruction. Pan et al. reduced the rest-volume of the tetrahedron via a tissue removal simulation, thereby simplifying the mesh reconstruction[10]. Lu et al. proposed a dual-mesh dynamic triangulation algorithm for mesh updates when tissues were removed[11].

2.3 Haptic rendering

Haptic rendering can be categorized into collision detection and collision response procedures. Mesh-based collision detection uses a level-of-detail strategy to accelerate collision detection between a virtual tool and a high-resolution mesh[12]. Voxel-based collision detection was used to simulate drill operations on virtual bones[13]. Constraint-based collision response is widely used for haptic rendering. These methods attempt to constrain the pose of the graphic tool to be free of penetration, whereas the haptic tool can penetrate objects. Compared with the penalty-based approach[14], constraint-based methods can yield more stable and accurate haptic results. Wang et al. proposed a series of constraint-based methods for six-DOF haptic rendering[15–17]. A sphere tree was used to fit the shape geometry and accelerate collision detection. Configuration-based optimization was performed to preserve the graphic tool by inserting an object. An overview of haptic rendering is presented in[18].

2.4 Real-time surgery simulation

Real-time surgery simulations with haptic feedback have been extensively investigated, of which laparoscopic surgery is a typical example. Sui et al. simulated electrocautery procedures for laparoscopic rectal cancer radical surgery[19]. Kim et al. presented a new method of deformable mesh carving for a laparoscopic cholecystectomy training simulator[20]. Li et al. proposed a patient-specific surgery PCI simulation system with wire and catheter physical simulations, as well as X-ray imaging simulations[21]. Ruthenbeck et al. proposed an endoscopic sinus surgery simulation method[22]. Fann et al. proposed a cardiac surgery simulation environment[23]. Wang et al. proposed a haptic-based dental simulator for preliminary user training[24]. Shi et al. proposed a simulation method for parenchyma delineation and splitting in virtual liver surgery[25], but only the per-defined region could be segregated.

3 Methods

3.1 Overview

An overview of our system is shown in Figure 1, which includes graphical and physical **modeling** of the liver, liver **topology updating**, and **physical simulation** with haptic feedback.

The modeling procedure includes the construction of a graphical model, physical model, and tetrahedral structure. We first reconstructed the coarse triangular meshes of the liver surface and intrahepatic vessels from CT data, using commercial software such as Mimics and 3ds Max. Subsequently, these meshes were optimized into isotropic meshes, the triangles of which were regular and had similar areas. Next, the

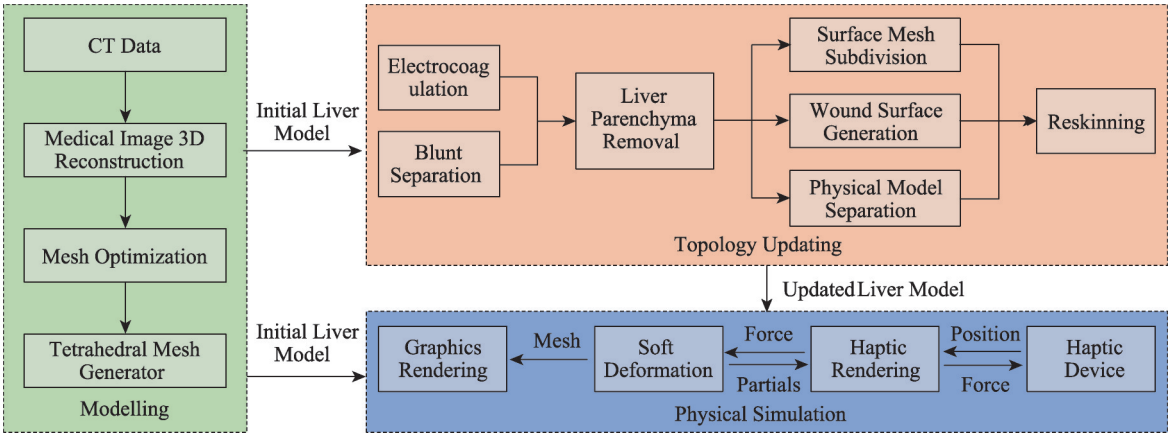

Figure 1 Overview of system.

internal space of the optimized meshes was discretized using nearly regular tetrahedrons, and the vertices (physical particles) of the tetrahedral mesh were used directly for soft deformation.

The topology updating procedure reconstructs the liver model based on the tetrahedral structure when the partial liver parenchyma is removed. We simulated two types of typically used surgical procedures: blunt separation and electrocoagulation. Blunt separation involves the physical destruction of the liver parenchyma, in which physical particles are removed directly. Electrocoagulation burns the liver parenchyma via a high-frequency current, and the physical particles are removed after heat conduction and phase change. The physical particles near the electrocoagulation hook were heated, and energy was transferred to other particles through the tetrahedral edges via a thermal transmission model. If the temperature of the physical particles reaches the threshold, the physical particles are removed, and the topology of the surface mesh and clusters must be updated. We segregated the surface triangle and generated a triangle on the wound surface based on an implicit tetrahedral structure. The corresponding clusters were separated based on the number of connected components.

During the physical simulation, soft deformation was performed in the position-based dynamic framework with a shape-matching constraint at a lower frequency (>30Hz). Subsequently, haptic rendering was updated at a higher frequency (>900Hz) in a different thread. The haptic rendering cycle comprised two phases: collision detection and collision response. Cluster-based shape matching was performed to accelerate the collision detection between virtual tools and physical particles, and a constraint-based optimization method was used to obtain the non-penetrating position of the graphical tools.

3.2 Three-dimensional model of liver system

The graphical meshes of the liver system include the liver capsule (the surface of the liver) and intrahepatic vessels (bile duct, hepatic artery, hepatic vein, and portal vein), as shown in Figure 2. Each graphical mesh is driven by a set of physical particles for soft deformation. As shown in Figure 3, different mesh skinning schemes are used for the liver capsule and intrahepatic vessels. Each vertex of the liver capsule is directly

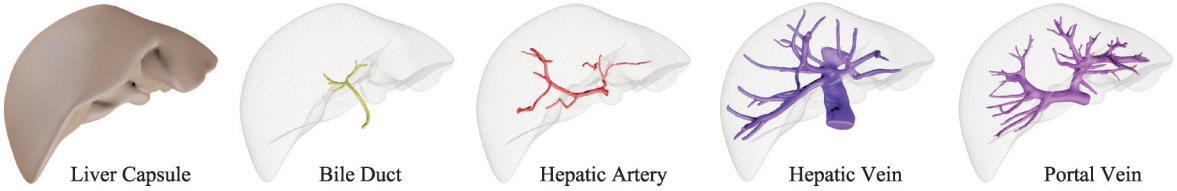

Figure 2 Graphical meshes of liver system.

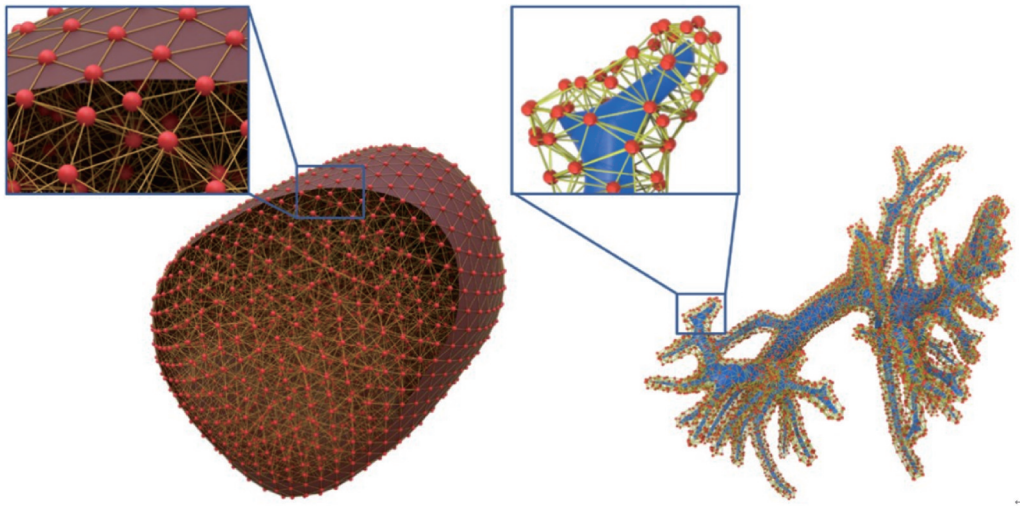

Figure 3 Illustration of liver model. Red spheres represent physical particles. Yellow lines are tetrahedral edges. Left shows sectional view of liver surface mesh and physical particles, which are simplified for visualization. Each vertex of liver mesh is directly driven by a physical particle during physical simulation. Right shows surface mesh and physical particles of intrahepatic vessels (only the portal vein is shown for clarity). Surface mesh of portal vein is embedded in the tetrahedral mesh. Every vertex of the portal vein mesh is driven by tetrahedron during physical simulation.

driven by a physical particle, while each vertex of the intrahepatic vessels is associated with a tetrahedron, and its position is calculated using the barycentric coordinates of the tetrahedron. In order to realize large-scale deformation and parallel acceleration, we use a position-based dynamics frame with a cluster-based shape-matching constraint. Specifically, the physical particles are grouped into several clusters of the same radius. The adjacent clusters contained overlapping particles, preventing liver destruction. Furthermore, we added additional clusters to physically connect the liver parenchyma and intrahepatic vessels. The simulation of liver parenchyma disassociation typically involves frequent topological updates of the graphical and physical models. Hence, we used a tetrahedral structure built on the physical particles of the liver parenchyma.

A 3D liver model can be constructed based on the geometric shape of the liver system effectively. Currently, the most low-cost and effective method is to reconstruct the triangle mesh from CT data, using frequently used medical image-based modeling software such as Materialise Mimics[26] or 3D Slicer[27]. The reconstructed surface triangle meshes may have sharp edges, large-area triangles, or long narrow triangles, which are not conducive to mesh deformation and physical model construction. Hence, we converted these meshes into isotropic meshes, which implies that all the triangles of the converted mesh were regular and had similar areas. The isotropic conversion guarantees the smoothness of the deformed triangular mesh and uniform distribution of the physical particle. We used the open-source algorithm Instant Mesh[28] to complete the isotropic conversion. The commercial software ZBursh[29] achieved the same results.

Based on the isotropic triangle mesh, we built a physical model for the liver parenchyma and intrahepatic vessels. We partitioned the three-dimensional space inside the liver into tetrahedrons using the automatic mesh generator TetGen[30]. Subsequently, tetrahedron vertices were used as the physical particles of the liver parenchyma. In this procedure, each vertex of the liver capsule mesh has a unique connection with a tetrahedron vertex. The optimized liver surface mesh was isotropic, and its triangles had the same area; hence, the vertices of the tetrahedrons were uniformly distributed inside the liver. Due to the complex tubular shape, the intrahepatic vessels have many more vertices than the liver capsule. Therefore, the vertices of the intrahepatic vessel mesh cannot be used as physical particles. We generated external tetrahedral meshes for the intrahepatic vessels using NetGen[31] to obtain physical particles. Additionally,

we used a cluster-based shape-matching constraint. Physical particles should be grouped into shape-matching clusters. The clustering procedure was performed on the tetrahedron vertices in our study. For each object of the liver system, we clustered the particles based on the tetrahedral center and predefined cluster radius, and the adjacent clusters contained overlapping particles; more details regarding the clusters are available in [4]. Furthermore, we created additional clusters to physically connect the liver parenchyma and intrahepatic vessels.

3.3 Topology updating for liver parenchyma removal

The parenchyma was removed via blunt dissection or electrocoagulation. The blunt dissection directly removed the physical particles, and the electrocoagulation burned out the physical particles by heating them to melting point. We used a heat transfer function[9] to simulate the energy transmission between the physical particles of the liver parenchyma along the edges of the tetrahedral mesh. Once the particles were removed, the vertex and triangle mesh of the wound surface were generated, and the corresponding clusters were separated into sub-clusters.

When a portion of the physical particles is removed, the interior of the corresponding tetrahedrons is exposed, and the corresponding triangles of the liver surface mesh must be removed or separated into small triangles. We generated the interior triangles of a tetrahedron based on marching tetrahedral[32] (MT), which is an extension of marching cubes[33] (MC). Compared with MC and dual conditioning methods[34], the MT algorithm does not require voxels stored in a cubic grid. When the vertices of a tetrahedron are removed, we generated new vertices on the corresponding edges using the weight sum of the two endpoints. Subsequently, the inner triangles were obtained by connecting the new vertices based on the predefined configurations. Within the tetrahedron, 14 triangle configurations can be realized (2^4-2; if no vertex or all vertices are removed, the inner triangle will not be generated). Figure 4b−Figure 4d show three of the 14 configurations, and the remaining 11 are the rotated versions of the three configurations.

The concept of marching tetrahedra can be used to separate triangles on the liver surface. As shown in Figure 4, if one or two vertices are removed, the new vertices on the corresponding edges are generated by

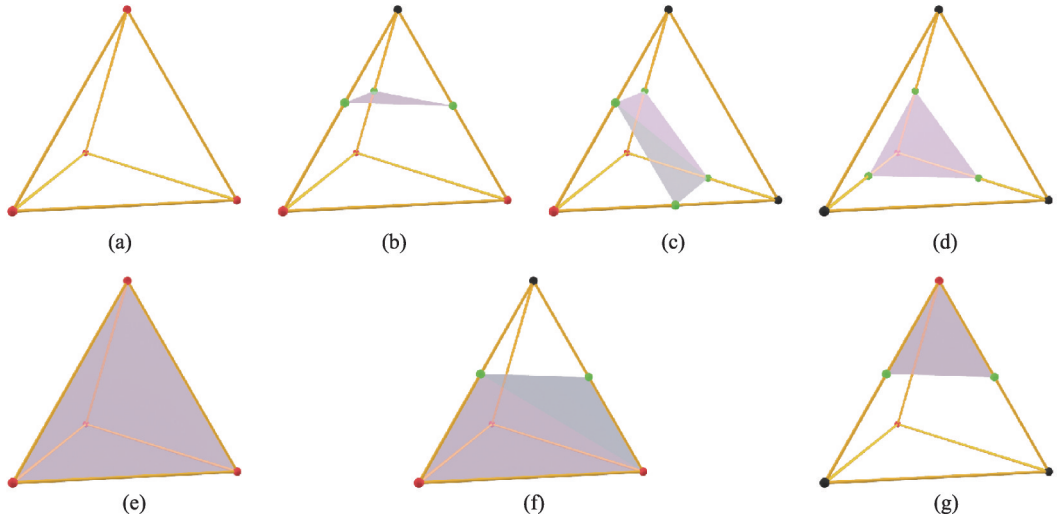

Figure 4 Inner triangle generation and surface triangle subdivision. Sphere in red implies valid state. Sphere in black means it is removed. Sphere in green is generated by weighed sum of two ends. (a) Original tetrahedron; (b) Triangle generation with one vertex removed; (c) Triangle generation with two vertices removed; (d) Triangle generation with three vertices removed; (e) Original triangle; (f) Triangle subdivision with one vertex removed; (g) Triangle subdivision with two vertices removed.

the weighted sum of the two endpoints. The triangle is separated into small triangles based on the predefined configurations. Six triangle configurations can be realized (2^3-2; if non-vertex or all vertices are removed, the triangle will not be generated). Figure 4f and Figure 4g show two of the six configurations, and the remaining four are the rotated versions of the two configurations.

When a portion of the physical particles is removed, a cluster may be separated into two disconnected sub-clusters without overlapping regions to achieve physical separation. The connected component detection[35] in graph theory can be used to separate clusters. The physical particles and tetrahedron edges can be regarded as an undirected graph, and the breadth-first search algorithm was applied for connected component detection. If only one connected component has been detected and no physical particles are completely removed, the cluster will not be updated, as shown in Figure 5b. If more than one connected component is detected, the clusters will be rebuilt from the connected components, as shown in Figure 5c.

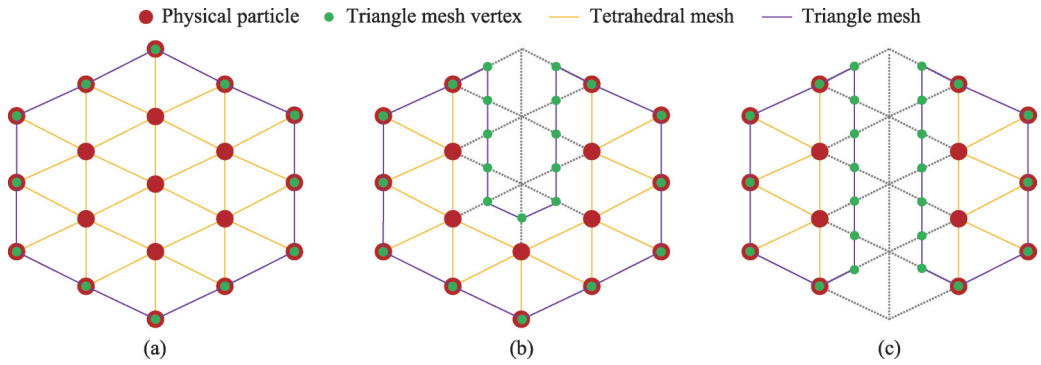

Figure 5 Cluster separation and mesh skinning. Red points indicate the physical particles and green points indicate the vertices of triangle mesh. (a) Original configuration of a cluster; (b) Several physical particles are removed from the cluster; (c) Several physical particles are removed and the cluster is divided into two sub-clusters.

As mentioned in Section 3.2, the vertices of the liver capsule mesh are directly driven by the physical particles. However, no physical particles are allocated to the vertices generated by marching tetrahedra. Therefore, we use the linear blending skinning method to obtain the position of a vertex$v$: $v = R_i p + T_i$, where i is the index of the nearest cluster to vertex v, p is the position in the local coordinate system of cluster i, R_i is the rotation matrix, and T_i is the translation vector.

3.4 Haptic rendering for virtual tool and soft body interaction

The popular haptic device can only provide 3-dimension force feedback at the current stage, and the torque force cannot be provided. Therefore, we provide two types of forces in our system: the contact force when the surgical instrument is pressed to the soft tissue and the pull force when grasping forceps are used to clip the soft tissue.

We used a constraint-based haptic rendering method to generate the contact force between the surgical instruments and deformable organs. To achieve this, we define three types of virtual tools in the haptic rendering cycle: haptic, physical, and graphical. As shown in Figure 6a, the position of the haptic tool is directly obtained from the haptic device, which is placed on the surface of the soft object via a non-penetration constraint when the haptic tool penetrates the virtual organ; the physical tool is used for collision detection, and its position is between the haptic and graphical tools. The positions of these tools are denoted as q_h^t, q_p^t and, q_g^t respectively, where t is the frame index.

The haptic rendering cycle comprised two phases: collision detection and collision response. In the collision-detection phase, the haptic tool might be inserted deeply into the soft object, as shown in Figure 6a. Therefore, we could not obtain particles near the object surface for graphical tool optimization. Hence,

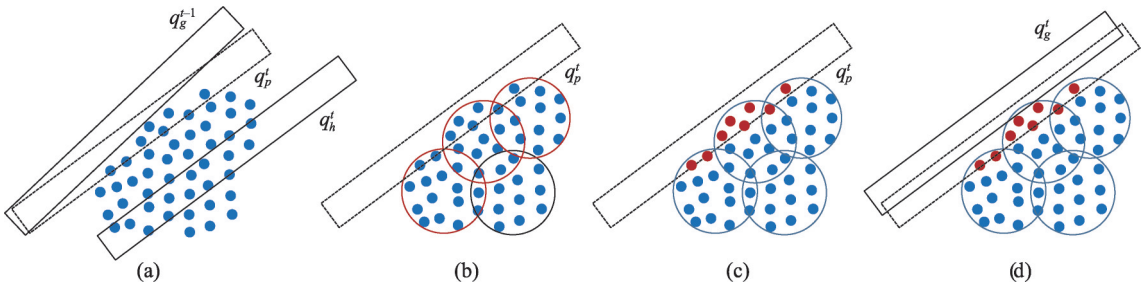

Figure 6 Haptic rendering for rigid tool and soft body interaction. (a) Definition of virtual tools; (b) collision detection with clusters; (c) collision detection with physical particles; (d) graphical tool optimization.

we defined a physical tool for collision detection using physical particles near the object surface. The position of the physical tool was calculated using the weighted sum of the position of the haptic tool, at frame *t,* and the position of the graphical tool in the last frame *t*-1, as follows: $q_p^t = \omega q_g^{t-1} + (1-\omega)q_h^t$. We typically set ω to 0.95, which causes the physical tool to penetrate partially into the object such that collision detection can be performed on the surface particles.

A parametric capsule was used to fit the graphic tool for the collision detection. The liver model typically contains tens of thousands of particles. Brute collision detection of these particles is time consuming. Hence, we accelerated collision detection using a two-step process based on the shape-matching clusters. The cluster radius and center were calculated for every simulation loop. The first step is capsule−sphere collision detection, as shown in Figure 6b. The second step is capsule−particle collision detection, as shown in Figure 6c. These procedures pertain to discrete collision detection; however, they are performed at a high frequency, and the tool passing through thin, soft objects can be effectively avoided.

In the collision response phase, we projected the physical tool to its non-penetration position (graphic tool) based on the particles obtained via collision detection. The position of the graphic tool $q_g = [x_g, y_g, z_g]$ was obtained by solving the following quadratic programming problem:

$$\begin{aligned} &\min_{q_g} \frac{1}{2}(q_g - q_h)^T K (q_g - q_h) \\ &\text{s.t.} C_1(q_g) \geqslant 0 \\ &\quad C_2(q_g) \geqslant 0 \\ &\quad \cdots \\ &\quad C_n(q_g) \geqslant 0 \end{aligned}$$

The quadratic term is the energy that represents the difference between the haptic and graphic tools, which implies that the graphic tool must be placed as close as possible to the haptic tool. In the above equation, K represents the stiffness matrix. The distance constraint $C_i(q) \geqslant 0$ implies that the graphic tool (capsule) cannot intersect with one of the physical particles i, and n is the number of particles colliding with the physical tool. The location of a physical particle is denoted by $p = [x_p, y_p, z_p]$. A particle may collide with a cylinder or two half-spheres of a capsule. If the particle collides with the cylinder of the capsule, the distance energy is defined as:

$$f(q) = \begin{cases} (D_c(p))^2 - R^2 & \text{collision with cylinder} \\ (D_s(p))^2 - R^2 & \text{collision with sphere} \end{cases}$$

Here, $D_c(p) = |(p-q) - dot(p-q, d_c)d_c|$ is the Euclidean distance between the particle and the cylinder axis, $D_s(p) = |p-q|^2$ is the Euclidean distance between the particle and sphere center, and R is the sum of the particle radius and capsule radius, where $d_c = [x_{dc}, y_{dc}, z_{dc}]$ is the unit vector of the capsule axis. A quadratic form was used to avoid square root operation. This is conducive to the gradient operation of

$f(q)$, which is a highly nonlinear formulation that cannot be solved effectively. It can be linearized using the first-order Taylor expansion, as follows:

$$f(q) \approx f(q^{t-1}) + \nabla f(q^{t-1})(q - q^{t-1})$$

where q^{t-1} is the position of the previous frame. The gradient operator $\nabla f(q^{t-1})$ is described in the Appendix. $C(q)$ can be expressed in the following linear form:

$$C(q) = \nabla f(q^{t-1})q - \nabla f(q^{t-1})q^{t-1} + f(q^{t-1})$$

The optimization problem in our study involved quadratic programming with linear inequality constraints. We used the open-source library Quadprog++[36,37] to solve this problem. Finally, the three-dimensional contact force at frame t was calculated by the difference vector between the haptic and graphical tools, as follows: $h_c^t = g(q_h^t - q_g^t)$, where g is the stiffness coefficient to control the force magnitude.

Grasping forceps are used to clip portions of the soft tissue. Consequently, we simulate this clipping operation by adding a hard position constraint to the particle in the grasping region; when the particles in the grasping region are detected, we first convert them to the local coordinate space of the surgical tool, and at every haptic frame, they are translated to the word position using the transition matrix of the haptic device. Based on these, the pull force h_p^t at frame t is generated by Hooke's law:

$$h_p^t = k\sum_{i=0}^{M}\sum_{j=0}^{N}\Delta d_{i,j}^t$$

where k is the stiffness coefficient, which was set to 0.001 in our system; M is the number of particles in the grasping region; N is the number of neighboring particles (we used the tetrahedral edge to find the neighboring particles). $\Delta d_{i,j}^t$ is the position offset between the clipped particle p_i and neighboring particle p_j:

$$\Delta d_{i,j}^t = \max(0, |p_i^t - p_j^t|^2 - |p_i^0 - p_j^0|^2)$$

When surgical instruments contact the organs, deformation should occur. To achieve this, the feedback force is used to change the velocity of the physical particles according to Newton's second law. Therefore, the mass of the physical particles or the stiffness of the clusters can be set to different values for different tissues.

4 Results

4.1 Hardware setup and experimental data

The hardware of our system was a workstation with an NVIDIA GeForce RTX 2080Ti, an Intel i9-9900K CPU, 16 GB RAM, and two haptic devices, as shown in Figure 7. The haptic device used was the Geomagic Touch[38], which can simulate the 6 DOF motion of surgical tools in practical surgery. The graphical meshes are shown in the first row of Figure 8. The clusters are visualized as spheres in the second row of Figure 8. The overlap region between the two clusters was equal to the cluster radius. Detailed information on the vertices, triangles, clusters, and tetrahedrons is presented in Table 1. The average cluster radius of the liver particles was set to 0.84cm and each cluster contained approximately 50 particles. This is in consideration of the efficiency trade-off between haptic collision and soft deformation.

Figure 7 Hardware setup.

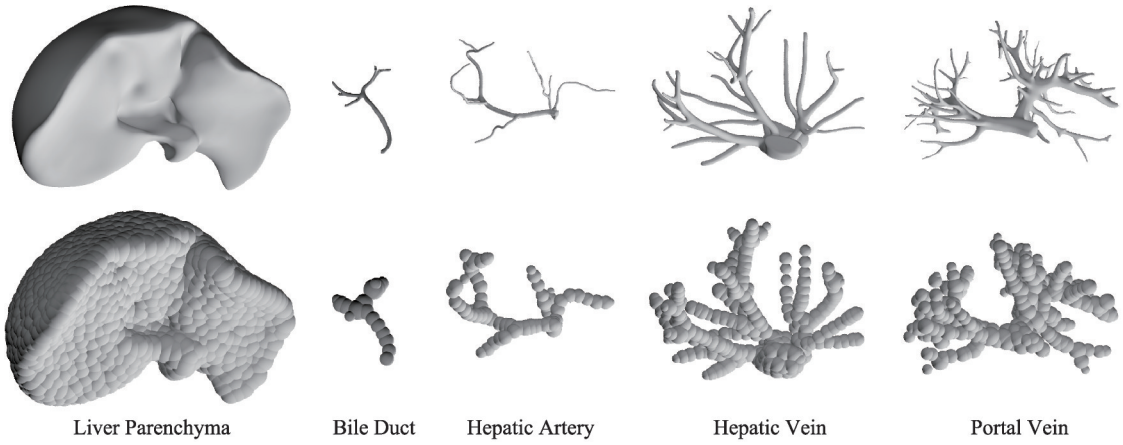

Figure 8 Visualization of the clusters. The top row shows the graphical meshes, and the bottom row shows the position and the radius of the clusters though spheres.

Table 1 Detailed geometrical information of liver system

	Vertex Amount	Triangle Amount	Tetrahedron Amount	Physical Particle Amount	Cluster Amount	Average Cluster Radius/cm
Liver	26721	53442	831932	146090	2994	0.84
Bile Duct	6825	13464	1644	5424	365	0.46
Hepatic Artery	13232	26460	1393	4010	434	0.47
Hepatic Vein	16300	32593	28263	8432	77	0.45
Portal Vein	22162	44318	21931	6898	22	0.47
Sum	85240	170277	885163	170854	3892	—

4.2 Simulation results

In this study, haptic rendering and soft deformation were developed using the C++ language and CUDA, respectively. In the simulation loop, soft deformation was simulated on the GPU in parallel. Liver parenchyma removal was simulated in a single CPU thread via haptic rendering, followed by mesh skinning of the liver, bile duct, hepatic artery, hepatic vein, and portal vein mesh was performed on independent CPU threads. Graphical rendering was developed using the Unity3D software. The virtual liver system contained more than 170k physical particles and 3.8k clusters. However, most of the physical particles and clusters were inside the liver, and they did not participate in haptic collision detection. Therefore, collision detection of haptic rendering was performed on the surface clusters to accelerate the haptic rendering cycle. We used the aforementioned parallel methods to guarantee the interactive performance of our system. Table 2 shows the average time cost for the sub-stepsof our system, and the time costs were determined based on the simulation scenes shown in Figure 9 and Figure 10.

Table 2 Average time cost of simulation cycle **ms**

Sub Steps	Soft Deformation	Haptic Rendering	Topology Update	Mesh Skinning & Graphics Rendering
Time Cost	36.8	1.05	98.3	20.83

Our system provides a realistic surgical scenario in terms of visual and force feedback. The simulation of the hepatic parenchymal removal process is shown in Figure 9 and Figure 10. Soft tissues were successively removed, and the topology of the wound surface mesh was updated naturally.

Figure 11 shows the interaction between the virtual tools and damaged liver. The physical model of the grasping forceps and scissors was approximated using two capsules for haptic rendering. During the

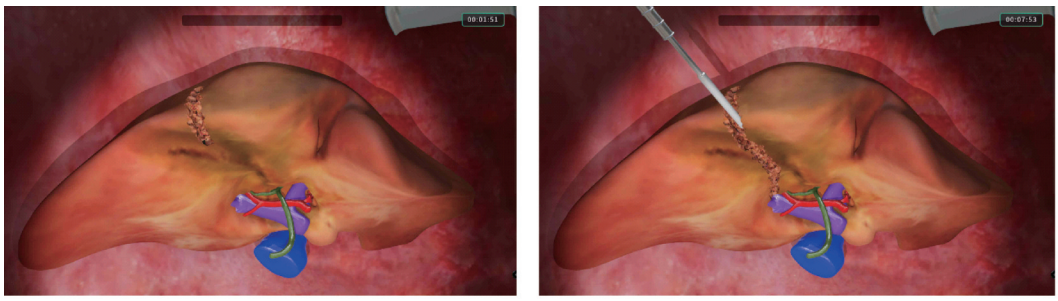

Figure 9 Electrocoagulation cauterization of hepatic parenchymal.

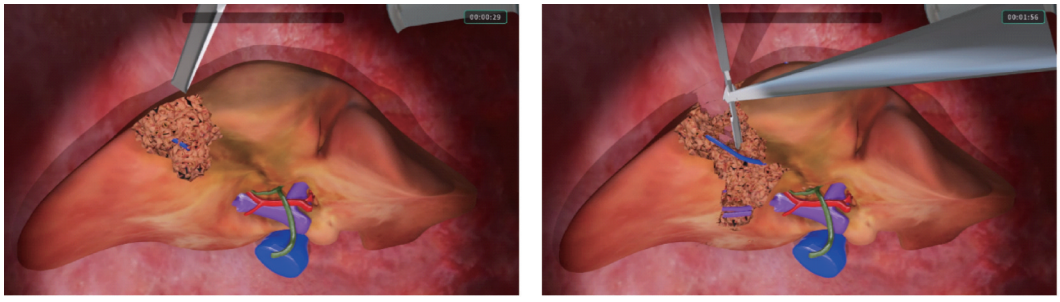

Figure 10 Blunt separation of hepatic parenchymal.

simulation, the constraint-based haptic rendering method can prevent the graphical tool from penetrating the virtual organ. The user can feel the virtual liver through the haptic handle when the surgical tools touch or grasp the soft tissue. The graphical tool position can be stably optimized at a high frequency (>900Hz).

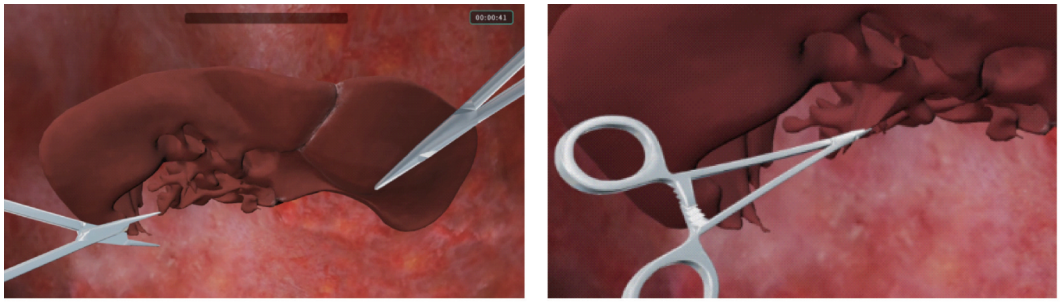

Figure 11 Haptic interaction between virtual tools and liver.

5 Discussion

In this study, we simulated parenchymal transection at an interactive rate. Our system integrates cluster-based shape matching, marching tetrahedral, and constraint-based haptic rendering. A tetrahedral mesh was used for physical and graphical topology updates. The clusters were used for soft deformation and haptic rendering acceleration. Furthermore, the system is accelerated by an asynchronous mechanism. Specifically, soft deformation, haptic rendering, topology update, mesh skinning, and graphics rendering were performed at different frequencies.

The shortcomings of our method are obvious. We only provide the operation procedure simulation of parenchymal transection, while the visual and tactile feedback cannot be provided precisely. We still need to measure the visual and tactile properties of real soft tissue to provide precise feedback. The reconstructed triangular mesh of the wound surface is not realistic enough because of the size of a single tetrahedron. When a tetrahedron is cut, the size of the generated triangle depends on the size of the tetrahedron. If the tetrahedron is rough, the generated mesh cannot represent the wound surface precisely.

The adaptive subdivision of the tetrahedron or triangle can be used to generate sharper cut edges. For the tetrahedrons containing the cut edge, smaller tetrahedrons should be generated and a finer triangle mesh should be constructed. The position-based dynamic framework of our system cannot guarantee physical correctness as the solving process highly depends on the iteration number, rather than on the physical properties of soft tissue. A feasible solution to this problem is the exploration of a real-time material point method. Volume deformation and rendering can be used for surgical simulation to achieve a more realistic visual effect. Hepatectomy is quite complex, and we only focus on parenchymal transection; thus, more surgical procedures can be simulated, such as the anatomy of the porta hepatis and separation of hepatic ligaments. These operations need to simulate the complex interaction between tools and veins of the liver, which refer to the rope simulation in the vein ligation procedure. Evaluation of surgical operation is an important aspect of the virtual surgery system. The data-driven evaluation of a certain procedure can be explored. Through data analysis, the procedure can be quantified by a data-driven system, which can train users to improve their surgical skills effectively by showing the operation procedure accurately.

Appendix

The gradient operator of the distance energy function at $q_g^{t-1} = [x_q, y_q, z_q]$ is expressed as:

$$\nabla f(q_g^{t-1}) = \left[\frac{df}{dx_q}, \frac{df}{dy_q}, \frac{df}{dz_q}\right]$$

If a particle collides with the cylinder of the capsule, the gradients are expressed as follows:

$$\frac{df}{dx_q} = 2(x_{pc} - x_p)(1 - x_{dc}x_{dc}) + 2(y_{pc} - y_p)(-x_{dc}y_{dc}) - 2(z_{pc} - z_p)(x_{dc}z_{dc})$$

$$\frac{df}{dy_q} = 2(x_{pc} - x_p)(-y_{dc}x_{dc}) + 2(y_{pc} - y_p)(1 - y_{dc}y_{dc}) - 2(z_{pc} - z_p)(y_{dc}z_{dc})$$

$$\frac{df}{dz_q} = 2(x_{pc} - x_p)(-z_{dc}x_{dc}) - 2(y_{pc} - y_p)(z_{dc}y_{dc}) + 2(z_{pc} - z_p)(1 - z_{dc}z_{dc})$$

where $d_c = [x_{dc}, y_{dc}, z_{dc}]$ is the unit vector of the capsule axis, $p_c = [x_{pc}, y_{pc}, z_{pc}]$ is the position of the particle projected on the cylinder axis, and $p = [x_p, y_p, z_p]$ is the location of the physical particles.

$$x_{pc} = x_q + dot(p - q_g^{t-1}, d_c)x_d$$

$$y_{pc} = y_q + dot(p - q_g^{t-1}, d_c)y_d$$

$$z_{pc} = z_q + dot(p - q_g^{t-1}, d_c)z_d$$

If a particle collides with the half-sphere of the capsule, the gradients are expressed as:

$$\frac{df}{dx_q} = 2\left(x_q - x_p\right)$$

$$\frac{df}{dy_q} = 2\left(y_q - x_p\right)$$

$$\frac{df}{dz_q} = 2\left(z_q - x_p\right)$$

Declaration of competing interest

We declare that we have no conflict of interest.

References

1 Yang T Y, Whitlock R S, Vasudevan S A. Surgical management of hepatoblastoma and recent advances. Cancers, 2019, 11(12): 1944
DOI:10.3390/cancers11121944

2 Delp S L, Loan J P, Basdogan C, Buchanan T S, Rosen J M. Surgical simulation: an emerging technology for military medical training. Proceedings of the National Forum: Military Telemedicine on-Line Today Research, Practice, and Opportunities, 1995, 29–34
DOI:10.1109/mtol.1995.504524

3 Müller M, Heidelberger B, Teschner M, Gross M. Meshless deformations based on shape matching. In: ACM SIGGRAPH 2005 Papers on-SIGGRAPH. Los Angeles, California, New York, ACM Press, 2005, 471–478
DOI:10.1145/1186822.1073216

4 Macklin M, Müller M, Chentanez N, Kim T Y. Unified particle physics for real-time applications. ACM Transactions on Graphics, 2014, 33(4): 1–12
DOI:10.1145/2601097.2601152

5 Provot X. Deformation constraints in a mass-spring model to describe rigid cloth behavior. Graphics Interface, 1995
DOI：10.1007/978-1-4471-0817-7_1

6 Bonet J, Wood R D. Nonlinear continuum mechanics for finite element analysis. Cambridge: Cambridge University Press, 2008
DOI:10.1017/cbo9780511755446

7 Müller M, Heidelberger B, Hennix M, Ratcliff J. Position based dynamics. Journal of Visual Communication and Image Representation, 2007, 18(2): 109–118
DOI:10.1016/j.jvcir.2007.01.005

8 Berndt I, Torchelsen R, Maciel A. Efficient surgical cutting with position-based dynamics. IEEE Computer Graphics and Applications, 2017, 37(3): 24–31
DOI:10.1109/mcg.2017.45

9 Pan J J, Yang Y H, Gao Y, Qin H, Si Y Q. Real-time simulation of electrocautery procedure using meshfree methods in laparoscopic cholecystectomy. The Visual Computer, 2019, 35(6/7/8): 861–872
DOI:10.1007/s00371-019-01680-z

10 Pan J J, Zhang L Y, Yu P, Shen Y, Wang H P, Hao H M, Qin H. Real-time VR simulation of laparoscopic cholecystectomy based on parallel Position-based dynamics in gpu. In: 2020 IEEE Conference on Virtual Reality and 3D User Interfaces (VR). Atlanta, GA, USA, IEEE, 2020, 548–556
DOI:10.1109/vr46266.2020.00076

11 Lu Z H, Arikatla V S, Han Z Q, Allen B F, De S. A physics-based algorithm for real-time simulation of electrosurgery procedures in minimally invasive surgery. The International Journal of Medical Robotics and Computer Assisted Surgery, 2014, 10(4): 495–504
DOI:10.1002/rcs.1561

12 Yoon S E, Salomon B, Lin M, Manocha D. Fast collision detection between massive models using dynamic simplification. In: Proceedings of the 2004 Eurographics/ACM SIGGRAPH symposium on Geometry processing-SGP. Nice, France, New York, ACM Press, 2004, 136–146
DOI:10.1145/1057432.1057450

13 Morris D, Sewell C, Barbagli F, Salisbury K, Blevins N H, Girod S. Visuohaptic simulation of bone surgery for training and evaluation. IEEE Computer Graphics and Applications, 2006, 26(6): 48–57
DOI:10.1109/mcg.2006.140

14 McNeely W A, Puterbaugh K D, Troy J J. Advances in voxel-based 6-DOF haptic rendering. In: ACM SIGGRAPH 2005 Courses on-SIGGRAPH. Los Angeles, California, New York, ACM Press, 2005
DOI:10.1145/1198555.1198606

15 Wang D X, Zhang X, Zhang Y R, Xiao J. Configuration-based optimization for six degree-of-freedom haptic rendering for fine manipulation. IEEE Transactions on Haptics, 2013, 6(2): 167–180
DOI:10.1109/toh.2012.63

16 Wang D X, Shi Y J, Liu S, Zhang Y R, Xiao J. Haptic simulation of organ deformation and hybrid contacts in dental operations. IEEE Transactions on Haptics, 2014, 7(1): 48–60
DOI:10.1109/toh.2014.2304734

17 Yu G, Wang D X, Zhang Y R, Xiao J. Simulating sharp geometric features in six degrees-of-freedom haptic rendering. IEEE Transactions on Haptics, 2015, 8(1): 67–78
DOI:10.1109/toh.2014.2377745

18 Wang D, Jiao J, Zhang Y, Zhao X. Computer haptics: haptic modeling and rendering in virtual reality environments. Journal of Computer-Aided Design & Computer Graphics, 2016, 28(6): 881−895
DOI:10.3969/j.issn.1003-9775.2016.06.003

19 Sui Y, Pan J J, Qin H, Liu H, Lu Y. Real-time simulation of soft tissue deformation and electrocautery procedures in laparoscopic rectal cancer radical surgery. The International Journal of Medical Robotics and Computer Assisted Surgery, 2017, 13(4): e1827
DOI:10.1002/rcs.1827

20 Kim Y, Kim L, Lee D, Shin S, Cho H, Roy F, Park S. Deformable mesh simulation for virtual laparoscopic cholecystectomy training. The Visual Computer, 2015, 31(4): 485−495
DOI:10.1007/s00371-014-0944-3

21 Li S, Xia Q, Hao A M, Qin H, Zhao Q P. Haptics-equiped interactive PCI simulation for patient-specific surgery training and rehearsing. Science China Information Sciences, 2016, 59(10): 103101
DOI:10.1007/s11432-016-0264-3

22 Ruthenbeck G S, Hobson J, Carney A S, Sloan S, Sacks R, Reynolds K J. Toward photorealism in endoscopic sinus surgery simulation. American Journal of Rhinology & Allergy, 2013, 27(2): 138−143
DOI:10.2500/ajra.2013.27.3861

23 Fann J I, Sullivan M E, Skeff K M, Stratos G A, Walker J D, Grossi E A, Verrier E D, Hicks G L, Feins R H. Teaching behaviors in the cardiac surgery simulation environment. The Journal of Thoracic and Cardiovascular Surgery, 2013, 145 (1): 45−53
DOI:10.1016/j.jtcvs.2012.07.111

24 Wang D X, Zhang Y R, Hou J X, Wang Y, Lv P, Chen Y G, Zhao H. iDental: a haptic-based dental simulator and its preliminary user evaluation. IEEE Transactions on Haptics, 2012, 5(4): 332−343
DOI:10.1109/toh.2011.59

25 Shi Y F, Liu M, Xiong Y S, Cai C, Tan K, Pan X H. The simulation of delineation and splitting in virtual liver surgery. In: 2015 International Conference on Virtual Reality and Visualization (ICVRV). Xiamen, China, IEEE, 2015, 264−268
DOI:10.1109/icvrv.2015.44

26 https://www.materialise.com/en/medical/mimics

27 https://www.slicer.org/

28 Jakob W, Tarini M, Panozzo D, Sorkine-Hornung O. Instant field-aligned meshes. ACM Transactions on Graphics, 2015, 34(6): 1−15
DOI:10.1145/2816795.2818078

29 https://zbrush.mairuan.com/

30 Si H. TetGen, a delaunay-based quality tetrahedral mesh generator. ACM Transactions on Mathematical Software, 2015, 41(2): 1−36
DOI:10.1145/2629697

31 https://ngsolve.org/

32 Muller H, Wehle M. Visualization of implicit surfaces using adaptive tetrahedrizations. In: Scientific Visualization Conference. Dagstuhl, Germany, IEEE, 1997, 243
DOI:10.1109/dagstuhl.1997.1423119

33 Lorensen W E, Cline H E. Marching cubes: a high resolution 3D surface construction algorithm. ACM SIGGRAPH Computer Graphics, 1987, 21(4): 163−169
DOI:10.1145/37402.37422

34 Ju T, Losasso F, Schaefer S, Warren J. Dual contouring of hermite data. ACM Transactions on Graphics, 2002, 21(3): 339−346
DOI:10.1145/566654.566586

35 https://en.wikipedia.org/wiki/Component_(graph_theory)

36 Goldfarb D, Idnani A. A numerically stable dual method for solving strictly convex quadratic programs. Mathematical Programming, 1983, 27(1): 1−33
DOI:10.1007/bf02591962

37 https://gitlab.inria.fr/alta/alta/-/tree/master/external/quadprog++

38 https://www.3dsystems.com/haptics-devices/touch/specifications